

## Mock Final Exam - Version V3

Total Points: 100

### Mock Final Exam - Version 3

Total Points: 100

Multiple Choice (12 questions, 3 points each = 36 points)

Select the best answer for each question.

1. (3 points) Which exception is specifically designed to be thrown when trying to read beyond the end of a file using classes like `DataInputStream` or `ObjectInputStream`?

- A. `IOException`
- B. `FileNotFoundException`
- C. `EOFException`
- D. `EndOfStreamException`

2. (3 points) When reading an object using `readObject()` from `ObjectInputStream`, what must you typically do with the returned `Object`?

- A. Immediately write it back to the stream to confirm.
- B. Cast it to its original, specific class type.
- C. Check if it's null.
- D. Convert it to a `String` using `toString()`.

3. (3 points) The `RandomAccessFile` class allows you to:

- A. Only read data sequentially from a file.
- B. Only write data sequentially to a file.

- C. Read and write data at arbitrary locations within a file.
- D. Access files randomly stored across different network locations.

4. (3 points) What is a potential disadvantage of using recursion compared to iteration for some problems?

- A. Recursive solutions are always harder to read.
- B. Recursion often uses more memory due to function call stack overhead.
- C. Recursion cannot solve problems involving loops.
- D. Recursive methods cannot return values.

5. (3 points) What is the output of the following recursive code?

```
public static void printNums(int n) {  
    if (n > 0) {  
        printNums(n - 1);  
        System.out.print(n + " ");  
    }  
}  
  
// Called with: printNums(3);
```

- A. 3 2 1
- B. 1 2 3
- C. 3
- D. 1 2 3 (with a trailing space)

6. (3 points) In the context of recursive directory traversal (like finding the total size), what serves as the base case?

A. Encountering a subdirectory.

B. Reaching the root directory.

C. Encountering a file (not a directory).

D. When the directory is empty.

7. (3 points) A recursive helper method is often used to:

A. Make the recursive method accessible outside its class.

B. Provide a simpler initial call signature for the user while the helper handles the actual recursion with more parameters (like index or accumulator).

C. Eliminate the need for a base case.

D. Ensure the recursion is tail-recursive.

8. (3 points) Which JavaFX class is typically used to represent the main window of an application?

A. Scene

B. Stage

C. Node

D. Window

9. (3 points) In JavaFX, property binding allows you to:

A. Link the value of one property directly to another, so changes in one automatically update the other.

B. Define the visual style properties (like color, font) using CSS.

C. Encrypt sensitive data stored in properties.

D. Handle exceptions related to property access.

10. (3 points) An abstract method is a method that:

A. Has an empty method body {}.

- B. Is declared static.
- C. Is declared final.
- D. Has no implementation and must be implemented by concrete subclasses.

11. (3 points) Why might a class implement the Cloneable interface?

- A. To indicate that its objects can be written to an ObjectOutputStream.
- B. To provide a standard way to compare its objects for ordering.
- C. To signal that it's permissible and meaningful to create a field-for-field copy of its objects.
- D. To ensure that the class can be used in JavaFX applications.

12. (3 points) Which Java I/O class should be used if you need to read character data from a file using the platform's default character encoding?

- A. FileInputStream
- B. FileReader
- C. DataInputStream
- D. BufferedReader

### Short Answer / Code Reading

1. (4 questions, variable points = 29 points)

2. (6 points) What is the difference between readChar() and readUTF() in DataInputStream?  
Mention the data type returned/read and how variable-length strings are handled.

3. (7 points) Consider the recursive method for calculating factorial:

```
public static long factorial(int n) {
```

```
if (n == 0) { // Base case
    return 1;
} else { // Recursive call
    return n * factorial(n - 1);
}
}
```

Trace the execution for factorial(3). Show the calls made and the values returned at each step until the final result is obtained.

4. (8 points) Explain the concepts of "event source," "event object," and "event handler" in the context of JavaFX event-driven programming. Give an example using a Button.

5. (8 points) You need to write a mixture of String (using writeUTF) and double values to a binary file. When reading this file back, why is it absolutely critical to read the data types in the exact same order they were written? What exception might occur if you try to read a double where a String was written, or vice-versa?

### Coding / Program Design

1. (4 questions, variable points = 35 points)

2. (10 points) Write a recursive Java method findMax(int[] arr, int index) that finds and returns the maximum value in the integer array arr starting from the given index to the end. Assume the array has at least one element. The initial call would be findMax(myArray, 0). You must use recursion. Define the base case and recursive step clearly within your code using comments.

```
public static int findMax(int[] arr, int index) {  
  
    // Base Case: ??  
  
    // Recursive Step: ?? (Compare arr[index] with result of recursive call)  
  
}
```

3. (12 points) Write a Java program snippet that uses `RandomAccessFile` to:

- Open a file named "sensor\_log.dat" in read-write ("rw") mode.
- Assume the file contains records, each consisting of an int (sensor ID) followed by a double (value).
- Move the file pointer to the beginning of the 3rd record (index 2).
- Read the int ID and double value from that record.
- Overwrite only the double value of that 3rd record with a new value (e.g., 99.9).
- Include basic try-catch for `IOException`.

4. (13 points) Design an interface named `Auditable` with a single abstract method `getLastAuditTimestamp()` that returns a `long`. Then, create a concrete class `FinancialRecord` that implements `Auditable`. `FinancialRecord` should have private instance variables for `recordId` (`String`) and `lastAudit` (`long`). Implement the constructor for `FinancialRecord` and the required `getLastAuditTimestamp()` method.

```
interface Auditable {  
  
    // Method signature?  
  
}
```

```
class FinancialRecord implements Auditable {  
  
    // Instance variables?
```

```
// Constructor?
```

```
// Implemented method?
```

```
}
```